

Proiect

Limbaje Formale si Compilatoare

Crearea unui compilator/translator

folosind FLEX si Bison

Modificarea exemplului 1_calculator

Tudor Dan-Mihail,

Informatica, grupa 40316

Continut:

- Programe folosite: Flex si Bison
- Funcționalitate si mod de utilizare
- Fișierul .l
- Fișierul .y
- Bibliografie

Programele folosite: Flex si Bison

FLEX este un generator de analizatoare lexicale. Acesta generează codul C pentru un analizator lexical (scanner) pe baza unui fișier de specificații .l creat de utilizator.

Fișierul .l conține un set de reguli de forma expresie regulata – acțiune. Expresiile regulate se folosesc pentru a grupa șirurile de caractere citite în *tokeni*, cărora li se vor asocia valori numerice. Pentru fiecare expresie se formează automatul finit corespunzător care o recunoaște.

BISON generează codul C pentru un analizator sintactic (parser), folosind un set de reguli dat de utilizator într-un fișier .y. Acest set de reguli descrie de fapt reguli gramaticale pentru a analiza *tokenii* primiți de la FLEX și a-i organiza într-un arbore sintactic.

Arborele sintactic va fi traversat depth first, în acest timp generându-se codul. Unele compilatoare pot produce direct cod executabil, în vreme ce altele au drept rezultat programul translatat în limbaj de asamblare.

Utilizarea Flex si Bison:

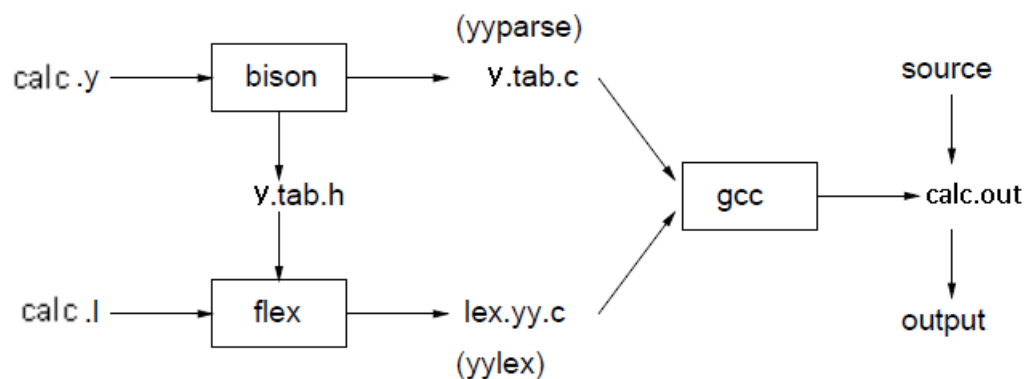


Fig1: succesiunea de operații implicate în realizarea unui compilator folosind FLEX și BISON

BISON citește descrierea gramaticii din fișierul `calc.y` și generează parserul (și anume funcția `yyparse()`) în fișierul `y.tab.c` și definițiile *tokenilor* în `y.tab.h`.

FLEX citește descrierile pattern-urilor din `calc.l` și creează pe baza regulilor descrise aici scanner-ul, descris de funcția `yylex`, conținută în fișierul `lex.yy.c`.

În final, parser-ul și scanner-ul sunt compilate împreună pentru a forma executabilul `calc.out`. Din funcția `main` se cheamă `yyparse` pentru a rula compilatorul. Acesta cheamă `yylex` pentru a obține următorul *token* din fișierul de intrare.

Translatorul se creează prin următoarele comenzi:

```
bison calc.y // creează y.tab.h și y.tab.c
```

```
flex calc.l // creează lex.yy.c
```

```
gcc -o calc.out y.tab.c lex.yy.c // compilare; se creează calc.out
```

Functionalitate si mod de utilizare

Calc.out va rula in terminal. Acesta permite atribuirea de valori unor variabile de un singur caracter literal, rezolvarea expresiilor folosind aceste variabile si verificarea valorii de adevăr a unor afirmații.

Operații permise:

Atribuire: $a=5$; $b=7$; $c=(6/2)$;

Incrementare/decrementare: $a++$; $a--$;

Operații aritmetice de baza: adunare, scădere, înmulțire, împărțire

Puteri: a^b (a la puterea b)

Modulo: $a\%b$ (restul împărțirii lui a la b, rezultatele vor fi valide doar pentru intregi, altfel se va afișa 0)

Radical: $\text{root}(a,b)$ (radical de ordin b din a)

Funcții trigonometrice: $\sin(a)$, $\cos(a)$, $\text{tg}(a)$, $\arcsin(a)$, $\arccos(a)$, $\text{arctg}(a)$

Funcții de convertire: $\text{DegtoRad}(a)$ – conversie grade-radiani

$\text{RadtoDeg}(a)$ – conversie radiani-grade

Comparație: $a>b$, $a<b$, $a\geq b$, $a\leq b$, $a==b$

Obținere rezultat: Rez: $a>b$; rez3: a/b ; (rez : $a>b$ va afișa valoarea `true` sau `false` a afirmației; Rez3 : a/b va afișa rezultatul expresiei cu numărul de zecimale precizat in instrucțiunea rez, in acest caz 3)

Instrucțiunile trebuie separate de `';`. Pe un rând se pot regăsi una sau mai multe instrucțiuni. In cazul mai multor acțiuni Rez pe același rând, rezultatele se vor afișa in ordinea scrierii, separate de un `'\n'`.

Erorile de sintaxa vor rezulta într-un mesaj ce specifica eroarea si in oprirea programului.

Fișierul .l

Fișierul **calc.l** conține trei secțiuni delimitate prin **%%**: definiții, reguli și subrutine.

Secțiunea de definiții:

```
1  %{
2  #include "y.tab.h"
3  #include <stdlib.h>
4  void yyerror(char *);
5  %}
```

Se include fișierul `y.tab.h` și biblioteca C `<stdlib.h>` (pentru a putea folosi funcțiile `atof()` și `strtol()`).

Secțiunea de reguli:

```
8  numbers ([0-9]+)|([0-9]*[.][0-9]+)|([0-9]+[.][0-9]*)
9  letter  [a-zA-Z]
10 pi      "pi"|"PI"|"Pi"
11 rezolvare ("rez"[0-9]?)|("Rez"[0-9]?)
12
13
14 %%
15 {letter} {
16     yylval.id = *yytext - 'a';
17     return VARIABLE;
18 }
19 {numbers} {
20     yylval.num = atof(yytext);
21     return NUMBER;
22 }
23 {pi} {
24     return PI;
25 }
26 {rezolvare} {
27     char *stop;
28     yylval.id = strtol(&yytext[3], &stop, 10);
29     if(yylval.id >= 10) yylval.id = 0;
30     return (REZ);
31 }
```

```

35  ▾  "+"      {
36                                     return INCR;
37                                     }
38  ▾  "--"     {
39                                     return DECR;
40                                     }
41  ▾  "=="     {
42                                     return EQ;
43                                     }
44  ▾  ">="     {
45                                     return GREQ;
46                                     }
47  ▾  "<="     {
48                                     return LSEQ;
49                                     }
50  ▾  "DegtoRad" {
51                                     return DEGTORAD;
52                                     }
53  ▾  "RadtoDeg" {
54                                     return RADTODEG;
55                                     }
56  ▾  "root"     {
57                                     return ROOT;
58                                     }
59  ▾  "sin"      {
60                                     return SIN;
61                                     }
62  ▾  "cos"      {
63                                     return COS;
64                                     }

```

Sectiunea de subrutine:

```
81  %%  
82  int yywrap(void) {  
83      return 1;  
84  }
```

Funcția `yywrap()` verifica daca fișierul de input a ajuns la final.

Fisierul .y

Fisierul **calc.y** va conține secțiunile definiții, reguli si subrutine.

Secțiunea definiții:

```
1  %{
2  #include <stdio.h>
3  #include <math.h>
4  #include <string.h>
5  #define YYERROR_VERBOSE 1
6  void yyerror(char *);
7  int yylex(void);
8  double sym[52];
9  %}
10
```

In codul de mai sus:

- Includem bibliotecile C necesare
- Instrucțiunea #define YYERROR_VERBOSE 1 se va asigura ca erorile returnate de yyerror() vor fi mai specifice
- Se defineste functia yylex
- Se declara vectorul sym care va retine valorile variabilelor;

```
11 %union
12 {
13     int id;
14     char bool[5];
15     double num;
16 }
17
18 %token <num> NUMBER
19 %token <id> VARIABLE PI REZ INCR DECR GREQ LSEQ EQ DEGTORAD RADTODEG ROOT SIN COS TAN ASIN ACOS ATAN
20 %nterm <num> stmt expression trig stmts
21 %nterm <bool> check
22 %left '+' '-' '*' '/' '^' '>' '<' ';' ',' ':' '%'
```

In codul de mai sus se definesc tipurile de date cu care se va lucra(integer, double si char) si se vor specifica tipurile *tokenilor* si neterminalilor.

Secțiunea de reguli:

```
25 v program: program stmts '\n'
26 | /* NULL (lambda) */
27 ;
28 v stmt: REZ ':' expression {
29 | printf("%.*f\n", $1, $3);
30 | }
31 | VARIABLE '=' expression { sym[$1] = $3; }
32 | VARIABLE INCR {sym[$1] ++;}
33 | VARIABLE DECR {sym[$1] --;}
34 | REZ ':' check { printf("%s\n", $3); }
35 ;
36 v stmts:
37 | stmt ';' stmts { $$ = $3; }
38 ;
```

O instructiune introdusa poate duce la:

- Rezolvarea unei expresii
- Initializarea unei variabile cu o anumita valoare
- Incrementarea sau decrementarea unei variabile
- Rezultatul valorii de adevar a unei afirmatii

Una sau mai multe instructiuni pot fi introduse pe un rand, adaugandu-se ';' la finalul fiecareia.

```
39 expression: NUMBER
40 | VARIABLE { $$ = sym[$1]; }
41 | VARIABLE INCR { $$ = sym[$1] + 1; sym[$1] ++; }
42 | VARIABLE DECR { $$ = sym[$1] - 1; sym[$1] --; }
43 | trig { $$ = $1; }
44 | '-' expression { $$ = -$2; }
45 | expression '+' expression { $$ = $1 + $3; }
46 | expression '%' expression {
47 | int ival1 = (int)$1, ival2 = (int)$3;
48 | if (ival1 == $1 && ival2 == $3)
49 | {
50 |     $$ = ival1 % ival2;
51 | }
52 | else
53 | {
54 |     printf("Values must be integers.\n");
55 |     $$ = 0;
56 | }
57 | }
58 | expression '-' expression { $$ = $1 - $3; }
59 | expression '*' expression { $$ = $1 * $3; }
60 | expression '/' expression { $$ = $1 / $3; }
61 | expression '^' expression { $$ = pow($1, $3); }
62 | '(' expression ')' { $$ = $2; }
63 ;
```

O expresie poate avea valoarea: unui numar, unei variabile, unei operatii de incrementare/decrementare(va returna valoarea variabilei +/- 1 si va incrementa/decrementa variabila), unei functii trigonometrica, unei negatii, unei operatii aritmetice sau a unei expresii intre paranteze.

Functii trigonometrice:

```
70  trig:  PI { $$ = M_PI; }
71      | DEGTORAD '(' expression ')' { $$ = $3 * M_PI / 180; }
72      | RADTODEG '(' expression ')' { $$ = $3 * 180 / M_PI; }
73      | ROOT '(' expression ',' expression ')' { $$ = pow($3, 1/$5); }
74      | SIN '(' expression ')' { $$ = sin($3); }
75      | COS '(' expression ')' { $$ = cos($3); }
76      | TAN '(' expression ')' { $$ = tan($3); }
77      | ASIN '(' expression ')' { $$ = asin($3); }
78      | ACOS '(' expression ')' { $$ = acos($3); }
79      | ATAN '(' expression ')' { $$ = atan($3); }
80      ;
```

Verificarea valorii de adevar:

```
64  check: expression '>' expression {if($1 > $3) strcpy($$, "true"); else strcpy($$, "false");}
65      | expression '<' expression {if($1 < $3) strcpy($$, "true"); else strcpy($$, "false");}
66      | expression GREQ expression {if($1 >= $3) strcpy($$, "true"); else strcpy($$, "false");}
67      | expression LSEQ expression {if($1 <= $3) strcpy($$, "true"); else strcpy($$, "false");}
68      | expression EQ expression {if($1 == $3) strcpy($$, "true"); else strcpy($$, "false");}
69      ;
```

Sectiunea de subrutine:

```
82  void yyerror(char *s) {
83      fprintf(stderr, "%s\n", s);
84  }
85
86  int main(void) {
87      yyparse();
88      return 0;
89  }
```

Se defineste functia `yyerror()` §

In main se va apela `yyparse()` ;

Bibliografie:

- 1_Introducere in FLEX si BISON.pdf (link pe ls.upg-elearning.ro)
- 2_Generatorul de analizatoare lexicale FLEX (link pe ls.upg-elearning.ro)
- 3_Generatorul de analizatoare sintactice BISON (link pe ls.upg-elearning.ro)